

Morse Code Translation Project Report

Prepared for Project Development

August 9, 2025

Problem Statement: The objective of this project is to develop a system for translating Morse code to English text and vice versa, with advanced features to enhance usability and functionality. The project will be implemented in three key steps:

1. **English to Morse Code Translation and Vice Versa:** Create a system to accurately translate English text (letters A-Z, numbers 0-9, and select punctuation) to Morse code and Morse code to English text.
2. **Recommending Similar Corrected Words:** For invalid Morse code inputs, suggest four possible English words based on similarity to valid Morse sequences, allowing the user to select a word and receive its correct Morse code.
3. **Input from Live Camera or Video:** Enable the system to capture Morse code input from a live camera or video feed by detecting visual signals (e.g., flashes of light representing dots and dashes).

Contents

1	Project Overview	2
2	Data Sources	2
3	Prior Work	2
4	Topics to Learn	3
5	Implementation Steps	3
6	Possible Issues and Challenges	4
7	Sample Implementation	6
8	Additional Considerations	9
9	Conclusion	10

1 Project Overview

This project aims to create a robust Morse code translation system that converts English text to Morse code and vice versa, recommends corrections for invalid Morse inputs, and accepts visual input from a live camera or video. Morse code encodes text as sequences of short signals (dots, `.`) and long signals (dashes, `-`). The system will be user-friendly, potentially with a command-line or graphical interface, and will incorporate advanced features like word suggestions and visual input processing to enhance functionality.

2 Data Sources

The project relies on the following data sources:

- **International Morse Code Chart:** Maps English letters (A-Z), numbers (0-9), and punctuation to Morse code sequences (e.g., A: `.-`, B: `-...`, 1: `.---`, `..`: `..-.`).
- **Public Domain Resources:** Websites like Wikipedia or electronics tutorials provide Morse code tables.
- **Existing Libraries:** Python libraries like `morsecode` or `morse-talk` offer predefined mappings.
- **Word List for Recommendations:** A list of common English words (e.g., from `words.txt` or Python's `words` module) for suggesting corrections.
- **Visual Input Data:** Video or live camera feeds capturing light signals (e.g., LED flashes) for Morse code detection, requiring image processing libraries.

No external datasets are required for translation, but a word list and video processing capabilities are needed for the new features.

3 Prior Work

Morse code translation is a common programming exercise. Existing implementations include:

- **Online Tools:** Websites like morsecode.world offer web-based translation.
- **Open-Source Projects:** GitHub hosts translators in Python, Java, and JavaScript, including command-line, GUI, and web apps.
- **Educational Resources:** Platforms like Codecademy feature Morse code exercises.
- **Embedded Systems:** Projects using Arduino for LED or audio Morse output.
- **Visual Morse Detection:** Some advanced projects use computer vision (e.g., OpenCV) to detect Morse code from light signals, though this is less common.

The word recommendation and visual input features are novel additions, requiring string similarity metrics and computer vision techniques.

4 Topics to Learn

Before starting, master the following:

1. Programming Fundamentals:

- Variables, data types, control structures.
- Dictionaries for mappings.
- String manipulation (splitting, joining, case conversion).

2. Morse Code Basics:

- International Morse Code standard.
- Rules: dots (.), dashes (-), spaces (letters), slashes (words).

3. Input/Output Handling:

- Reading and validating user input.
- Formatting output.

4. Error Handling and Recommendations:

- Managing invalid inputs.
- String similarity algorithms (e.g., Levenshtein distance).
- User selection from recommendations.

5. Computer Vision for Visual Input:

- Image processing with libraries like OpenCV.
- Detecting light signals (e.g., brightness changes) in video frames.
- Converting visual signals to Morse code sequences.

6. User Interface (Optional):

- Command-line or GUI frameworks (e.g., Tkinter, HTML/JavaScript).
- Real-time video display for camera input.

7. Optional Advanced Topics:

- Regular expressions for validation.
- Audio generation (e.g., `pygame`).
- Web development (e.g., Flask, React).

5 Implementation Steps

The project will be implemented in three steps:

1. English to Morse Code Translation and Vice Versa:

- Create a dictionary for letters (A-Z), numbers (0-9), and punctuation.

- For English to Morse: Convert input to uppercase, map characters to Morse, join with spaces (letters) and slashes (words), and handle invalid characters.
 - For Morse to English: Split input by spaces and slashes, use reverse mapping, and handle invalid sequences with recommendations.
2. **Recommending Similar Corrected Words:**
- For invalid Morse sequences, calculate similarity (e.g., Levenshtein distance) to valid Morse sequences.
 - Map similar sequences to English words from a word list.
 - Suggest four possible words and allow user selection to output the correct Morse code.
3. **Input from Live Camera or Video:**
- Use a library like OpenCV to capture video frames from a camera or video file.
 - Detect light signals (e.g., LED flashes) by analyzing brightness changes.
 - Convert signals to Morse code (short flashes as dots, long flashes as dashes).
 - Integrate with the translation system to process detected Morse code.
4. **Input Validation:** Validate English (A-Z, 0-9) and Morse (., -, /, spaces) inputs, and ensure robust video signal detection.
5. **Build User Interface:**
- Command-line: Prompt for translation direction, input, and word selection.
 - GUI (optional): Use Tkinter or web frameworks, with video display for camera input.
6. **Test the System:** Test translations (e.g., “SOS” $\leftrightarrow$ “... — ...”), recommendations (e.g., “.-. .-.. —” suggesting “FLOOR”, “SLOPE”), and video input accuracy.
7. **Add Optional Features:** Include audio output, file I/O, or translation history.
8. **Document and Deploy:** Provide documentation and deploy (e.g., GitHub Pages).

6 Possible Issues and Challenges

Implementing the Morse code translation system with the specified features presents several potential issues and challenges:

- **Input Validation Complexity:**
 - *Challenge:* Ensuring robust validation for diverse inputs (text, Morse, and video signals) can be complex, especially with ambiguous or malformed inputs (e.g., mixed case text, non-standard Morse separators, or noisy video signals).
 - *Solution:* Use regular expressions for text and Morse validation, and implement adaptive thresholds or noise filtering for video signal detection.

- **Word Recommendation Accuracy:**
 - *Challenge:* Generating relevant word suggestions for invalid Morse inputs requires a comprehensive word list and efficient similarity metrics. Small word lists may lead to irrelevant suggestions, and complex algorithms (e.g., Levenshtein distance) can be computationally expensive.
 - *Solution:* Use a large dictionary (e.g., `/usr/share/dict/words`) and optimize similarity calculations by precomputing Morse codes for common words or using approximate matching techniques.
- **Video Signal Detection Reliability:**
 - *Challenge:* Detecting Morse code from video (e.g., light flashes) is sensitive to lighting conditions, camera quality, and background noise. Distinguishing dots from dashes based on duration can be error-prone in inconsistent environments.
 - *Solution:* Calibrate brightness thresholds dynamically, use frame differencing to reduce noise, and implement temporal analysis to accurately measure signal durations. Test under varied lighting conditions.
- **Real-Time Processing Performance:**
 - *Challenge:* Processing live video feeds in real-time requires significant computational resources, especially for high-resolution video or low-power devices, potentially causing delays or dropped frames.
 - *Solution:* Optimize video processing with downscaled frames, efficient algorithms (e.g., using NumPy for brightness calculations), and hardware acceleration (e.g., GPU support in OpenCV).
- **User Interface Complexity:**
 - *Challenge:* Integrating text input, word recommendations, and video display into a cohesive GUI can be complex, especially ensuring accessibility and responsiveness across platforms (e.g., desktop, web).
 - *Solution:* Use modular frameworks (e.g., Tkinter for desktop, React for web) and test for accessibility features like keyboard navigation and screen reader compatibility.
- **Cross-Platform Compatibility:**
 - *Challenge:* Ensuring the system works across different operating systems and devices (e.g., webcams on Windows vs. Linux) may encounter hardware-specific issues or library incompatibilities.
 - *Solution:* Use cross-platform libraries like OpenCV and test on multiple environments. Provide fallback options (e.g., manual input if video capture fails).
- **Dependency Management:**
 - *Challenge:* The project relies on external libraries (e.g., `python-levenshtein`, `opencv-python`), which may introduce installation or version compatibility issues.

- *Solution:* Document dependencies clearly, use virtual environments, and test with specific library versions to ensure reproducibility.

7 Sample Implementation

Below is a Python command-line implementation including translation, word recommendations, and basic video input processing using OpenCV:

Listing 1: Morse Code Translator with Recommendations and Video Input

```

1 # Required imports
2 import Levenshtein
3 import cv2
4 import numpy as np
5
6 # Sample word list (use a larger dictionary in practice)
7 WORD_LIST = ['HELLO', 'WORLD', 'FLOOR', 'SLOPE', 'CLOSE', 'HOUSE',
8              ]
9
10 # Morse code dictionary
11 MORSE_CODE_DICT = {
12     'A': '.-', 'B': '-...', 'C': '-.-.', 'D': '-..', 'E': '.',
13     'F': '..-', 'G': '--.', 'H': '....', 'I': '..', 'J': '.---',
14     'K': '-.-', 'L': '.-..', 'M': '--', 'N': '-.', 'O': '---',
15     'P': '..-.', 'Q': '--.-', 'R': '.-.', 'S': '...', 'T': '-',
16     'U': '..-', 'V': '...-', 'W': '.--', 'X': '-.-.-', 'Y': '-.-.-',
17     'Z': '--..', '0': '-----', '1': '.-----', '2': '..-----', '3': '...--',
18     '4': '....-', '5': '.....', '6': '-....', '7': '--...', '8': '---..',
19     '9': '----.', '.': '.-.-.-', ',': '--.-.-', '?': '..-.-.'
20 }
21
22 # Reverse dictionary for Morse to English
23 MORSE_TO_ENGLISH = {value: key for key, value in MORSE_CODE_DICT.items()}
24
25 def english_to_morse(text):
26     """Convert English text to Morse code."""
27     text = text.upper()
28     morse_words = []
29     for word in text.split():
30         morse_chars = []
31         for char in word:
32             if char in MORSE_CODE_DICT:
33                 morse_chars.append(MORSE_CODE_DICT[char])
34             else:
35                 raise ValueError(f"Invalid character: {char}")
36         morse_words.append(' '.join(morse_chars))
37     return ' / '.join(morse_words)

```

```

37
38 def get_word_suggestions(invalid_code, word_list, morse_dict, n
    =4):
39     """Suggest n possible words based on invalid Morse code."""
40     suggestions = []
41     for word in word_list:
42         try:
43             morse = english_to_morse(word)
44             distance = Levenshtein.distance(invalid_code, morse)
45             suggestions.append((word, morse, distance))
46         except ValueError:
47             continue
48     suggestions.sort(key=lambda x: x[2]) # Sort by distance
49     return [(word, morse) for word, morse, _ in suggestions[:n]]
50
51 def morse_to_english(morse):
52     """Convert Morse code to English text with suggestions for
        invalid input."""
53     words = morse.split('␣/␣')
54     english_words = []
55     for word in words:
56         chars = word.split()
57         english_chars = []
58         for char in chars:
59             if char in MORSE_TO_ENGLISH:
60                 english_chars.append(MORSE_TO_ENGLISH[char])
61             else:
62                 print(f"Invalid␣Morse␣code:␣{char}")
63                 suggestions = get_word_suggestions(word,
64                     WORD_LIST, MORSE_CODE_DICT)
65                 if suggestions:
66                     print("Possible␣words:")
67                     for i, (sug_word, sug_morse) in enumerate(
68                         suggestions, 1):
69                         print(f"{i}.␣{sug_word}␣({sug_morse})")
70                     choice = input("Select␣a␣word␣(1-4,␣or␣0␣to␣
71                         skip):␣")
72                     if choice.isdigit() and 1 <= int(choice) <=
73                         len(suggestions):
74                         selected_word = suggestions[int(choice)
75                             -1][0]
76                         english_chars = list(selected_word)
77                         return english_to_morse(selected_word) #
78                         Return selected word's Morse
79                     else:
80                         raise ValueError("Skipping␣invalid␣
81                             sequence")
82                 else:
83                     raise ValueError("No␣suggestions␣available")
84     english_words.append(''.join(english_chars))
85     return ''.join(english_words)

```

```

79
80 def detect_morse_from_video(video_source=0):
81     """Detect Morse code from live camera or video using OpenCV.
82         """
83     cap = cv2.VideoCapture(video_source)
84     morse_code = []
85     signal_times = []
86     prev_brightness = 0
87     start_time = None
88     frame_count = 0
89     fps = cap.get(cv2.CAP_PROP_FPS) or 30 # Assume 30 FPS if
90     unknown
91
92     while cap.isOpened():
93         ret, frame = cap.read()
94         if not ret:
95             break
96         # Convert to grayscale and calculate average brightness
97         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
98         brightness = np.mean(gray)
99
100         # Detect signal changes (threshold-based)
101         if brightness > 100 and prev_brightness <= 100: # Light
102             on
103             start_time = frame_count / fps
104         elif brightness <= 100 and prev_brightness > 100: #
105             Light off
106             if start_time is not None:
107                 duration = (frame_count / fps) - start_time
108                 if duration < 0.2: # Short signal (dot)
109                     morse_code.append('.')
110                 else: # Long signal (dash)
111                     morse_code.append('-')
112                 signal_times.append(duration)
113
114         prev_brightness = brightness
115         frame_count += 1
116         cv2.imshow('Video_Feed', frame)
117         if cv2.waitKey(1) & 0xFF == ord('q'):
118             break
119
120     cap.release()
121     cv2.destroyAllWindows()
122     return ''.join(morse_code)
123
124 def main():
125     """Main function for the command-line interface."""
126     while True:
127         print("\nMorse_Code_Translator")
128         print("1. English_to_Morse")
129         print("2. Morse_to_English")

```



```

126     print("3. Morse from Camera/Video")
127     print("4. Exit")
128     choice = input("Choose an option (1-4): ")
129
130     if choice == '1':
131         text = input("Enter English text: ")
132         try:
133             result = english_to_morse(text)
134             print(f"Morse code: {result}")
135         except ValueError as e:
136             print(f"Error: {e}")
137     elif choice == '2':
138         morse = input("Enter Morse code (use spaces between
139             letters, / between words): ")
140         try:
141             result = morse_to_english(morse)
142             print(f"English text: {result}")
143         except ValueError as e:
144             print(f"Error: {e}")
145     elif choice == '3':
146         print("Starting video capture (press 'q' to stop) ...")
147         morse = detect_morse_from_video() # Use 0 for webcam
148         or provide video file path
149         if morse:
150             print(f"Detected Morse code: {morse}")
151             try:
152                 result = morse_to_english(morse)
153                 print(f"English text: {result}")
154             except ValueError as e:
155                 print(f"Error: {e}")
156         else:
157             print("No Morse code detected.")
158     elif choice == '4':
159         print("Exiting...")
160         break
161     else:
162         print("Invalid choice. Try again.")
163
164 if __name__ == "__main__":
165     main()

```

8 Additional Considerations

- **Scalability:** Optimize for large inputs or real-time video processing using efficient algorithms.
- **Extensibility:** Support additional Morse standards or character sets (e.g., Cyrillic).

- **Recommendation Accuracy:** Use larger word lists or advanced similarity metrics.
- **Video Processing Robustness:** Improve signal detection with adaptive thresholds or noise filtering.
- **Accessibility:** Ensure GUI interfaces support keyboard navigation or screen readers.
- **Testing:** Test translations, recommendations, and video input accuracy (e.g., detecting “... — ...” from flashes).

9 Conclusion

This project provides a comprehensive opportunity to practice programming, data structures, user interface design, and computer vision. The system supports English-Morse translation, word recommendations for invalid inputs, and visual input from a camera or video, with careful consideration of potential challenges. The provided Python implementation serves as a foundation, which can be extended with a GUI, audio output, or other features.